

DOKUMENTASI PROYEK RE-BOOK

"Diajukan untuk memenuhi Ujian Akhir Semester pada mata kuliah
Teknologi dan Aplikasi Mobile (TAM)"

Dosen Pengampu: Agung Pambudi, S.Kom., M.Kom.



Disusun Oleh:

Kharisma Jaka Harum	(2417051068)
Elsy Aliffia Sirony Putri	(2417051025)
Viandra Thridya Agsatri	(2417051008)
Ruddy Haristianata	(2457051011)

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS LAMPUNG

2026

DAFTAR ISI

DAFTAR ISI.....	2
1. PENDAHULUAN.....	3
1.1 Arsitektur Utama.....	3
1.2 Sumber Data Utama.....	3
2. ALUR NAVIGASI ANTAR HALAMAN.....	5
2.1 Halaman Awal.....	5
2.2 Daftar Rute dan Tujuannya.....	5
2.3 ViewModel yang Digunakan.....	5
3. MENU EXPLORE.....	6
3.1 Inisialisasi Data saat Halaman Explore Dibuka.....	6
3.2 Mekanisme Alur Proses Pencarian Buku.....	6
3.3 Mekanisme Pengambilan dan Penampilan Gambar Sampul Buku.....	7
3.4 Mekanisme Navigasi ke Halaman Detail Buku.....	7
3.5 Ringkasan Arsitektur Data Menu Explore.....	7
4. MENU BUKU SAYA.....	8
4.1 Mekanisme Identifikasi Kepemilikan Buku Berdasarkan Sesi Pengguna.....	8
4.2 Implementasi Query dan Pemantauan Data pada Room Database.....	8
4.3 Mekanisme Perubahan Status Buku.....	8
4.4 Mekanisme Penghapusan Data Buku.....	9
4.5 Ringkasan Arsitektur Data Menu Buku Saya.....	9
5. MENU PROFIL.....	10
5.1 Perbedaan DataStore VS Room Database.....	10
5.2 Mekanisme Aktivasi Mode Edit Profil.....	10
5.3 Mekanisme Pembaruan Data Profil.....	10
5.4 Mekanisme Pengakhiran Sesi Pengguna.....	10
5.5 Ringkasan Arsitektur Data Menu Profil.....	11
6. MENU UPLOAD BUKU.....	12
6.1 Mekanisme Pengambilan Metadata Buku dari OpenLibrary API.....	12
6.2 Mekanisme Identifikasi Pemilik Buku Berdasarkan Sesi Pengguna.....	12
6.3 Proses Penyimpanan dan Publikasi Data Buku.....	13
6.4 Ringkasan Arsitektur Data Menu Upload Buku.....	13
7. MENU WISHLIST.....	14
7.1 Sifat Penyimpanan.....	14
7.2 Mekanisme Manajemen Daftar Wishlist Baru.....	14
7.3 Mekanisme Alur Proses Membuka Menu Wishlist.....	15
7.4 Mekanisme Navigasi ke Halaman Detail Buku dari Wishlist.....	15
7.5 Ringkasan Sumber Data Wishlist.....	15
8. RINGKASAN PERBANDINGAN SELURUH MENU.....	16
8.1 Catatan Tambahan.....	16

1. PENDAHULUAN

Re-Book adalah aplikasi Android berbasis Jetpack Compose yang berfungsi sebagai salah satu platform berbagi, donasi, dan pinjam buku antar mahasiswa. Aplikasi ini dibangun untuk mengikuti pola arsitektur MVVM (Model-View-ViewModel) dengan dua sumber data utama, yaitu basis data lokal (Room Database) yang tersimpan di perangkat, dan layanan API publik OpenLibrary yang diakses melalui koneksi internet.

Dokumen ini disusun untuk menjelaskan secara rinci dari mana setiap data yang ditampilkan pada tiap menu atau fitur berasal, bagaimana proses pengambilan data tersebut bekerja, serta bagaimana alur navigasi antar halaman berlangsung ketika pengguna berinteraksi dengan elemen-elemen pada antarmuka.

1.1 Arsitektur Utama

Aplikasi ini disusun dalam tiga lapisan utama yang saling berkomunikasi secara satu arah, yaitu lapisan tampilan (UI/Screen), lapisan logika (ViewModel), dan lapisan data (Repository, Database, dan API). Pemisahan ini bertujuan agar kode lebih mudah dikelola, diuji, dan dikembangkan.

Lapisan	Komponen	Tanggung Jawab
UI/Screen	ExploreScreen, MyBookScreen, ProfileScreen, UploadScreen, WishlistScreen, dan sebagainya	Menampilkan data ke pengguna dan menangkap interaksi seperti saat klik tombol dan input teks.
ViewModel	BookViewModel, ProfileViewModel, LoginViewModel, dan sebagainya	Menyimpan state UI, menjalankan proses bisnis, dan menjembatani Screen dengan Repository.
Repository & Data	BookRepository, AuthRepository, Room Database, OpenLibrary API	Sumber data baik dari internet maupun dari penyimpanan lokal perangkat.

1.2 Sumber Data Utama

Hal terpenting yang perlu dipahami sebelum membaca alur tiap fitur adalah bahwa aplikasi ini memiliki dua sumber data yang sama sekali berbeda sifatnya:

- ❖ Data dari internet (OpenLibrary API), dipakai untuk fitur pencarian buku referensi umum di menu Explore dan fitur auto isi metadata di menu Upload. Data ini tidak disimpan permanen, hanya ditampilkan sementara hasil pencarian.
- ❖ Data dari database lokal (Room/SQLite), dipakai untuk menyimpan buku yang

diupload pengguna (tabel books) dan data akun pengguna (tabel users). Data ini tersimpan secara permanen di perangkat selama aplikasi tidak dihapus datanya.

Selain itu, terdapat pula menu Wishlist yang sumber datanya adalah memori sementara aplikasi (in-memory state) yaitu tidak tersimpan ke database maupun API, sehingga akan hilang ketika aplikasi ditutup paksa.

2. ALUR NAVIGASI ANTAR HALAMAN

Seluruh perpindahan antar halaman (screen) diatur secara terpusat pada satu file bernama NavGraph.kt menggunakan komponen NavHost dari pustaka Jetpack Navigation Compose. Setiap halaman didaftarkan dengan sebuah nama rute (route), dan perpindahan dilakukan dengan memanggil `navController.navigate("nama_rute")`.

2.1 Halaman Awal

Aplikasi secara otomatis memeriksa status sesi login pengguna melalui SessionManager. Jika belum ada sesi yang tersimpan, pengguna akan diarahkan ke halaman Login. Jika sudah pernah login dan sesi masih tersimpan, pengguna langsung diarahkan ke halaman Dashboard tanpa perlu login ulang.

```
startDestination = if (authToken == null) "login" else "dashboard"
```

2.2 Daftar Rute dan Tujuannya

Rute	Halaman	Dapat Dituju Dari
login	LoginScreen	Pembuka aplikasi
register	RegisterScreen	Tombol “Daftar Sekarang” di halaman login
dashboard	DashboardScreen	Setelah login berhasil
explore	ExploreScreen	Kartu menu “Explore” di dashboard
detail	BookDetailScreen	Klik salah satu buku di Explore atau Wishlist
upload	UploadScreen	Kartu menu “Upload” di dashboard
profile	ProfileScreen	Kartu menu “Profil” di dashboard
wishlist	WishlistScreen	Kartu menu “Wishlist” di dashboard
my books	MyBooksScreen	Kartu menu “Buku Saya” di dashboard

2.3 ViewModel yang Digunakan

Perlu dicatat bahwa BookViewModel dibuat satu kali di NavGraph dan dipakai bersama (shared) oleh enam halaman sekaligus: Dashboard, Explore, BookDetail, Upload, Wishlist, dan MyBooks. Karena menggunakan instance yang sama, perubahan data pada satu halaman misalnya menambahkan buku ke Wishlist dari halaman Detail akan langsung terlihat di halaman Wishlist tanpa perlu memuat ulang data.

3. MENU EXPLORE

Menu Explore berfungsi sebagai fitur pencarian buku referensi dari sumber terbuka di internet. Pengguna dapat mengetikkan judul buku atau nomor ISBN untuk mencari informasi buku.

Sumber data: Layanan OpenLibrary API (<https://openlibrary.org/search.json>), data diambil langsung dari internet secara real-time, bukan dari database lokal aplikasi.

3.1 Inisialisasi Data saat Halaman Explore Dibuka

Begitu BookViewModel pertama kali dibuat (saat aplikasi pertama kali masuk ke salah satu dari enam halaman yang menggunakannya), terdapat blok inisialisasi yang otomatis menjalankan pencarian dengan kata kunci bawaan “science”. Inilah sebabnya menu Explore sudah menampilkan daftar buku meskipun pengguna belum mengetik apa pun di kolom pencarian.

```
init { searchBooks("science") }
```

3.2 Mekanisme Alur Proses Pencarian Buku

1. Pengguna mengetik kata kunci pada kolom pencarian (OutlinedTextField). Setiap karakter yang diketik langsung memperbarui variabel searchQuery di memori sementara halaman tersebut.
2. Tombol “Cari” hanya muncul apabila kolom pencarian tidak kosong. Saat tombol ini ditekan, fungsi viewModel.searchBooks(searchQuery) dipanggil.
3. Di dalam BookViewModel, status pencarian (_booksState) langsung diubah menjadi ApiResult.Loading pada tahap ini, layar otomatis menampilkan ikon loading berputar (CircularProgressIndicator).
4. BookViewModel meneruskan permintaan ke BookRepository.searchBooks(). Repository ini akan memeriksa apakah teks yang dicari berupa nomor ISBN (10 atau 13 digit angka) atau judul biasa, lalu menyusun alamat URL pencarian yang sesuai.
5. Repository membuka koneksi internet (HttpURLConnection) ke server OpenLibrary, menunggu respons berupa data JSON, lalu mengurai (parsing) data tersebut satu per satu menjadi daftar objek Book.
6. Hasil pencarian dikembalikan sebagai ApiResult.Success berisi daftar buku, atau ApiResult.Error apabila terjadi gangguan jaringan atau waktu tunggu (timeout) habis.
7. Halaman ExploreScreen yang terus “mengamati” status booksState melalui collectAsState() otomatis memperbarui tampilannya begitu hasil pencarian tersedia baik menampilkan daftar buku, pesan “tidak ditemukan”, atau pesan kegagalan beserta tombol “Coba Lagi”.

3.3 Mekanisme Pengambilan dan Penampilan Gambar Sampul Buku

Gambar sampul (cover) setiap buku tidak disimpan di dalam aplikasi maupun di database. Gambar dimuat langsung dari URL milik OpenLibrary setiap kali halaman ditampilkan, menggunakan pustaka Coil melalui komponen AsyncImage.

URL: <https://covers.openlibrary.org/b/id/{coverId}-M.jpg>

Apabila data buku dari API tidak memiliki ID sampul (cover_i bernilai kosong), aplikasi menampilkan kotak placeholder berwarna abu-abu dengan ikon buku sebagai gantinya.

3.4 Mekanisme Navigasi ke Halaman Detail Buku

Ketika sebuah kartu buku (BookCard) diklik, fungsi `onBookClick(book)` dijalankan. Pada `NavGraph`, callback ini berisi dua aksi: pertama menyimpan buku yang dipilih ke dalam `viewModel.selectBook(book)`, kedua melakukan navigasi ke halaman detail (`navController.navigate("detail")`). Halaman `BookDetailScreen` kemudian membaca buku yang tersimpan tersebut melalui `viewModel.selectedBook`.

3.5 Ringkasan Arsitektur Data Menu Explore

Elemen Tampilan	Sumber Data	Cara Memperoleh
Daftar judul, penulis, tahun terbit	OpenLibrary API	HTTP GET ke <code>search.json</code> , hasil JSON diurai ke objek <code>Book</code> .
Gambar sampul buku	OpenLibrary Covers	URL gambar dibentuk dari <code>cover_i</code> , dimuat oleh <code>Coil AsyncImage</code> .
Status loading/error	State internal ViewModel	Nilai sealed class <code>ApiResponse</code> yang diubah selama proses <code>fetch</code> .
Badge status buku (Tersedia/Dipinjam/Donasi)	Field status pada hasil API	Selalu default <code>AVAILABLE</code> karena buku hasil pencarian API bukan milik siapa pun di aplikasi.

4. MENU BUKU SAYA

Menu Buku Saya menampilkan seluruh buku yang telah diunggah oleh pengguna yang sedang login pada perangkat tersebut, beserta fitur untuk mengubah status ketersediaan dan menghapus buku.

Sumber data: Room Database lokal (tabel books), data diambil dari penyimpanan internal perangkat, bukan dari internet.

4.1 Mekanisme Identifikasi Kepemilikan Buku Berdasarkan Sesi Pengguna

Daftar buku yang ditampilkan ditentukan oleh properti `myBooks` pada `BookViewModel`. Properti ini dibangun dengan cara mengikuti (observe) data sesi pengguna yang sedang login (`sessionManager.userData`), kemudian secara otomatis mengambil seluruh buku dari database yang kolom `ownerEmail`-nya sesuai dengan email pengguna tersebut.

```
val myBooks: StateFlow<List<Book>> = sessionManager.userData
    .flatMapLatest { user ->
        if (user != null) repository.getMyBooks(user.email)
        else flowOf(emptyList())
    }
```

Karena pendekatan ini bersifat reaktif, apabila pengguna logout kemudian login dengan akun lain, daftar yang ditampilkan akan otomatis berganti mengikuti akun yang baru tanpa perlu memuat ulang halaman secara manual.

4.2 Implementasi Query dan Pemantauan Data pada Room Database

```
@Query("SELECT * FROM books WHERE ownerEmail = :email ORDER BY
uploadedAt DESC")
```

```
fun getBooksByOwner(email: String): Flow<List<BookEntity>>
```

Hasil query ini berupa `Flow` artinya data ini terus dipantau secara langsung. Setiap kali ada perubahan pada tabel books (baik penambahan, pengeditan status, maupun penghapusan), tampilan pada layar akan otomatis menyesuaikan tanpa campur tangan pengguna.

4.3 Mekanisme Perubahan Status Buku

1. Pengguna menekan ikon pensil pada salah satu kartu buku (`MyBookCard`).
2. Variabel state lokal `bookToEdit` diisi dengan data buku yang dipilih, sehingga sebuah jendela dialog (`AlertDialog`) otomatis muncul berisi pilihan status: Tersedia, Dipinjam, atau Didonasikan.
3. Saat salah satu pilihan (`RadioButton`) dipilih, fungsi `viewModel.updateBookStatus(book.id, status)` langsung dipanggil.

4. Fungsi ini meneruskan permintaan ke BookRepository, yang menjalankan perintah SQL UPDATE books SET status = ... WHERE id = ... pada database.
5. Dialog otomatis tertutup, dan karena data dipantau secara reaktif melalui Flow, badge status pada kartu buku akan langsung berubah sesuai pilihan baru.

4.4 Mekanisme Penghapusan Data Buku

1. Pengguna menekan ikon tong sampah pada kartu buku.
2. Muncul dialog konfirmasi bertuliskan “Apakah Anda yakin ingin menghapus buku ‘(judul buku)’?” penghapusan tidak langsung terjadi pada tahap ini.
3. Apabila pengguna menekan tombol “Hapus” (berwarna merah) pada dialog, baru fungsi viewModel.deleteBook(book.id) dijalankan.
4. Perintah SQL DELETE FROM books WHERE id = ... dieksekusi di database, dan baris data buku tersebut hilang secara permanen dari tabel.
5. Karena data dipantau secara reaktif, kartu buku tersebut langsung lenyap dari daftar tanpa perlu memuat ulang halaman.

4.5 Ringkasan Arsitektur Data Menu Buku Saya

Elemen Tampilan	Sumber Data	Catatan
Daftar buku saya	Tabel books di Room Database, disaring berdasarkan ownerEmail	Hanya menampilkan buku yang diunggah lewat akun yang sedang login
Status buku	Kolom status pada tabel books	Dapat diubah langsung dari menu ini
Aksi edit status	UPDATE pada tabel books	Tersimpan permanen, tidak terhubung ke internet
Aksi hapus buku	DELETE pada tabel books	Tidak dapat dibatalkan setelah dikonfirmasi

5. MENU PROFIL

Menu Profil menampilkan data identitas pengguna yang sedang login, serta memungkinkan pengguna untuk mengubah nama, nomor WhatsApp, dan status (Mahasiswa/Alumni), juga menyediakan tombol untuk keluar dari sesi (logout).

Sumber data: DataStore Preferences (penyimpanan sesi lokal bernama user_session) bukan dari tabel users di Room Database, dan bukan dari internet.

5.1 Perbedaan DataStore VS Room Database

Perlu digarisbawahi bahwa data profil yang ditampilkan di menu ini bersumber dari SessionManager (DataStore), bukan langsung dari tabel users pada Room Database. DataStore berfungsi sebagai “salinan sesi aktif” yang disimpan saat pengguna berhasil login atau mendaftar, agar aplikasi tidak perlu terus-menerus melakukan query ke database hanya untuk mengetahui siapa yang sedang login.

```
var user: stateFlow<UserProfile?> = sessionManager.userData
    .stateIn(viewModelScope,      SharingStarted.WhileSubscribed(5000),
    null)
```

5.2 Mekanisme Aktivasi Mode Edit Profil

Menekan tombol ini hanya mengubah nilai state lokal isEditing dari false menjadi true. Tidak ada data yang berubah pada tahap ini, hal ini murni mengganti tampilan dari mode “lihat” ke mode “form edit”. Tiga kolom input (nama, nomor WA, status) sebelumnya sudah diisi otomatis dengan data pengguna yang sedang aktif.

5.3 Mekanisme Pembaruan Data Profil

1. Sistem memeriksa apakah panjang nomor WhatsApp berada pada rentang 10 hingga 13 digit. Apabila valid, fungsi viewModel.updateProfile(nama, telepon, status) dipanggil.
2. Fungsi ini diteruskan ke AuthRepository.updateProfile(), yang kemudian memanggil sessionManager.updateProfile() untuk menuliskan data baru ke dalam DataStore.
3. Karena objek user pada ViewModel bersifat reaktif terhadap perubahan DataStore, seluruh bagian aplikasi yang menampilkan data pengguna (termasuk salam “Halo, ...” pada Dashboard) akan otomatis ikut memperbarui tampilannya.
4. Tampilan kembali berpindah ke mode “lihat” (isEditing diubah menjadi false).

5.4 Mekanisme Pengakhiran Sesi Pengguna

1. Fungsi viewModel.logout() dipanggil, yang meneruskan perintah ke AuthRepository.logout().

2. Fungsi ini memanggil `sessionManager.clearSession()`, yang menghapus seluruh data (token, nama, email, status, dsb.) yang tersimpan di DataStore.
3. Callback `onLogoutClick()` dijalankan, yang pada NavGraph berisi perintah navigasi kembali ke halaman Login sekaligus membersihkan seluruh riwayat halaman (back stack) agar pengguna tidak dapat menekan tombol kembali untuk masuk ulang ke halaman yang membutuhkan status login.

5.5 Ringkasan Arsitektur Data Menu Profil

Elemen Tampilan	Sumber Data	Catatan
Nama, email, status, kontak WA	DataStore (SessionManager) lokal perangkat	Tersimpan sejak login/registrasi pertama kali.
Institusi	Teks tetap “Universitas Lampung”	Tidak diambil dari basis data mana pun.
Perubahan profil	Ditulis ulang ke DataStore	Tidak mengubah tabel users di Room Database.
Tombol logout	Menghapus seluruh data di DataStore	Tidak menghapus akun, hanya mengakhiri sesi.

6. MENU UPLOAD BUKU

Menu Upload memungkinkan pengguna menambahkan data buku baru ke dalam aplikasi agar dapat dilihat dan dipinjam oleh pengguna lain. Menu ini memiliki dua sumber data yang bekerja secara berurutan: pertama mengambil dari internet (opsional, untuk pengisian otomatis), kedua menyimpan ke database lokal (wajib, saat tombol simpan ditekan).

Sumber data: Tahap pencarian metadata: OpenLibrary API (internet). Tahap penyimpanan akhir: Room Database lokal (tabel books).

6.1 Mekanisme Pengambilan Metadata Buku dari OpenLibrary API

Pada bagian atas formulir terdapat kolom pencarian dengan ikon kaca pembesar. Fitur ini bertujuan membantu pengguna agar tidak perlu mengetik seluruh data buku secara manual.

1. Pengguna mengetikkan judul atau ISBN buku pada kolom pencarian metadata, lalu menekan ikon kaca pembesar.
2. Karena proses pengambilan data dari internet bersifat asynchronous, pemanggilan fungsi dibungkus dalam scope.launch (Coroutine) menggunakan rememberCoroutineScope().
3. Status isSearchingMetadata diubah menjadi true, sehingga ikon kaca pembesar untuk sementara digantikan oleh ikon loading kecil yang berputar.
4. Fungsi viewModel.fetchMetadata(query) dipanggil, yang di belakangnya memanggil repository.searchBooks(query, limit = 1) meminta satu hasil teratas saja dari OpenLibrary API.
5. Apabila data ditemukan, seluruh kolom formulir (judul, penulis, deskripsi, tahun terbit, kategori, dan URL sampul) langsung terisi otomatis sesuai hasil pencarian.
6. Apabila data tidak ditemukan, status isNotFound diubah menjadi true dan pesan pemberitahuan ditampilkan, mempersilakan pengguna mengisi data secara manual.

```
suspend fun fetchMetadata(query: String): Book? {  
    val result = repository.searchBooks(query, limit = 1)  
    result if (result is ApiResult.Success &&  
        result.data.isNotEmpty())  
        result.data.first() else null  
}
```

6.2 Mekanisme Identifikasi Pemilik Buku Berdasarkan Sesi Pengguna

Nama dan nomor WhatsApp pemilik buku tidak diketik manual oleh pengguna pada formulir ini. Data tersebut diambil secara otomatis dari sesi login yang sedang

aktif melalui SessionManager, sehingga setiap buku yang diunggah otomatis terhubung dengan identitas akun yang sedang digunakan.

6.3 Proses Penyimpanan dan Publikasi Data Buku

Tombol ini hanya akan aktif (enabled = true) apabila kolom Judul dan Penulis sudah terisi. Saat ditekan, langkah-langkah berikut terjadi secara berurutan:

1. Sebuah objek Book baru dibentuk, dengan id yang dibuat secara acak dan unik menggunakan `UUID.randomUUID().toString()` tanpa perlu berkomunikasi dengan server mana pun untuk menentukan ID tersebut.
2. Status buku secara default selalu diset `BookStatus.AVAILABLE` (Tersedia), karena buku yang baru diunggah belum pernah dipinjam siapa pun.
3. Fungsi `viewModel.uploadBook(newBook, currentEmail)` dipanggil.
4. Di dalam `BookRepository`, objek `Book` dikonversi menjadi `BookEntity` (dengan tambahan kolom `ownerEmail`), lalu disimpan ke database melalui `bookDao.insertBook()`.
5. Callback `onUploadSuccess()` dijalankan, yang pada `NavGraph` akan menavigasikan pengguna kembali ke halaman sebelumnya (umumnya Dashboard atau Buku Saya).

6.4 Ringkasan Arsitektur Data Menu Upload Buku

Elemen Tampilan	Sumber Data	Catatan
Hasil pencarian metadata otomatis	OpenLibrary API	Sifatnya opsional, dapat dilewati dan diisi manual
Nama & kontak WA pemilik	Sesi login aktif (SessionManager)	Diambil otomatis, tidak diketik manual
ID unik buku	Dibuat lokal di perangkat	Tidak memerlukan koneksi internet untuk membuatnya
Penyimpanan akhir buku	Tabel books pada Room Database lokal	Permanen tersimpan di perangkat, dapat dilihat di Buku Saya

7. MENU WISHLIST

Menu Wishlist menampilkan kumpulan buku yang telah ditandai sebagai favorit oleh pengguna untuk disimpan sebagai referensi pribadi.

Sumber data: Memori sementara aplikasi (in-memory state pada BookViewModel) tidak tersimpan ke Room Database maupun DataStore, dan tidak terhubung ke internet.

7.1 Sifat Penyimpanan

Berbeda dengan menu Buku Saya yang datanya tersimpan permanen di database, daftar Wishlist hanya disimpan pada variabel `_wishlist` bertipe `MutableStateFlow` di dalam `BookViewModel`, yang berarti data ini hanya bertahan selama aplikasi berjalan (selama proses aplikasi masih hidup di memori perangkat). Apabila aplikasi ditutup paksa (force close) atau perangkat dimatikan, daftar Wishlist akan kembali kosong saat aplikasi dibuka kembali.

```
private val _wishlist = MutableStateFlow<List<Book>>(emptyList())  
  
val wishlist: StateFlow<List<Book>> = _wishlist
```

7.2 Mekanisme Manajemen Daftar Wishlist Baru

Hal yang penting untuk dipahami adalah bahwa `MenuWishlistScreen.kt` sendiri tidak memiliki tombol untuk menambahkan buku. Tombol tambah (berbentuk ikon hati) berada pada halaman `BookDetailScreen`, yang dapat diakses dari menu Explore maupun dari Wishlist itu sendiri dengan mengklik salah satu buku.

1. Pengguna membuka detail sebuah buku (dari Explore atau dari Wishlist), lalu menekan ikon hati pada bagian atas halaman.
2. Fungsi `viewModel.toggleWishlist(book)` dipanggil.
3. Fungsi ini memeriksa apakah buku tersebut sudah ada di dalam daftar wishlist. Jika sudah ada, buku tersebut dihapus dari daftar (ikon hati berubah menjadi kosong). Jika belum ada, buku tersebut ditambahkan ke dalam daftar (ikon hati berubah menjadi terisi penuh berwarna merah).

```
fun toggleWishlist(book: Book) {  
    val current = _wishlist.value.toMutableList()  
    if (current.any { it.id == book.id }) {  
        current.removeAll { it.id == book.id }  
    } else {  
        current.add(book)  
    }  
    _wishlist.value = current  
}
```

7.3 Mekanisme Alur Proses Membuka Menu Wishlist

1. Halaman membaca daftar buku langsung dari `viewModel.wishlist` menggunakan `collectAsState()`, tidak ada proses pengambilan data tambahan (tidak ada koneksi internet, tidak ada query database) yang dijalankan saat halaman ini dibuka.
2. Apabila daftar kosong, ditampilkan ilustrasi emoji hati patah beserta pesan “Wishlist Anda masih kosong” sebagai pemberitahuan kepada pengguna.
3. Apabila daftar memiliki isi, ditampilkan dalam bentuk `LazyColumn` menggunakan komponen kartu `BookCard`, komponen tampilan yang sama persis dengan yang digunakan pada menu `Explore`.

7.4 Mekanisme Navigasi ke Halaman Detail Buku dari Wishlist

Sama seperti pada menu `Explore`, mengklik sebuah buku akan memanggil `viewModel.selectBook(book)` lalu menavigasikan pengguna ke halaman `BookDetailScreen` untuk melihat informasi lengkap buku tersebut.

7.5 Ringkasan Sumber Data Wishlist

Elemen Tampilan	Sumber Data	Catatan
Daftar buku wishlist	Memori sementara (in-memory) <code>BookViewModel</code>	Tidak permanen, hilang saat aplikasi ditutup paksa
Penambahan atau penghapusan wishlist	Dilakukan dari <code>BookDetailScreen</code>	Bukan dari halaman <code>Wishlist</code> itu sendiri
Tampilan kartu buku	Komponen <code>BookCard</code> yang sama dengan menu <code>Explore</code>	Reuse komponen, tidak ada kode duplikat

8. RINGKASAN PERBANDINGAN SELURUH MENU

Tabel berikut merangkum sumber data dari seluruh menu yang telah dijelaskan, untuk memudahkan pembaca melihat perbandingannya secara sekilas.

Menu	Sumber Data Utama	Tersimpan Permanen?	Butuh Internet?
Explore	OpenLibrary API	Tidak	Ya
Buku Saya	Room Database (tabel books)	Ya	Tidak
Profil	DataStore (SessionManager)	Ya	Tidak
Upload	OpenLibrary API (opsional) + Room Database (penyimpanan akhir)	Ya (setelah disimpan)	Hanya saat auto-fill
Wishlist	Memori sementara aplikasi (in-memory)	Tidak	Tidak

8.1 Catatan Tambahan

1. Data buku hasil pencarian Explore dan data buku yang diunggah lewat Upload disimpan dalam struktur (model) yang sama, yaitu kelas Book, namun melalui jalur penyimpanan yang berbeda.
2. Login dan Register sama-sama memproses data melalui AuthRepository, yang berinteraksi dengan tabel users pada Room Database untuk verifikasi, dan dengan DataStore untuk menyimpan sesi aktif setelah berhasil.
3. Seluruh proses yang melibatkan database atau jaringan internet dijalankan pada thread latar belakang (Dispatchers.IO) memanfaatkan Kotlin Coroutines, sehingga antarmuka aplikasi tidak mengalami hambatan (freeze) saat data sedang diproses.